LECTURE-19

Tuesday.
21/4/26

Banker's Algorithm:

- n = no. of processes
- m = no. of resources

- Available Vector = A vector of m size showing unallocated resources
- Max (given) Matrix = nxm matrix defines the maximum no. of resources needed by each process from start till end.
- Allocation = n x m matrix showing which resource is currently allocated to which process.
- Need Matrix = nxm matrix → Max matrix − Allocation Matrix.
  ↳ we'll find ourselves

⇒ Safety Algo: Algo to check whether a state is safe or not.
- It could be used independly or inside Banker's Algo too.

- Work Vector: A copy of "available" vector (size m) since values will keep changing.
- Finish Vector: Length 'n' vector of Boolean Type (initially False completely)

[STEPS]: (of safety)

① Find i such that Work = Available and Finish[i] == false.
Need_i <= Work. If no such i exist, go to step ③.
(jis process ka nikla, uski raw)

② Work = Work + Allocation and Finish[i] = True. Now go back to ①

③ If finish[i] == true for all i, then the system is in 'safe mode.'

NOTE: In Safety Algo; we aren't changing any states or giving any resources. We are just theoratically kay ager ye sab kuch krlein to it'll be a safe sequence. If found then we'll carry that process. else that req. will be declined.

Q: Safety Algo.

	Allocation			Max			Available		
	A	B	C	A	B	C	A	B	C
P0	0	1	0	7	5	3	3	3	2
P1	2	0	0	3	2	2			
P2	3	0	2	9	0	2			
P3	2	1	1	2	2	2			
P4	0	0	2	4	3	3			

Answer: Make a need matrix.

Need = Max - Allocation.

	Need		
	A	B	C
P_0	7	4	3
P_1	1	2	2
P_2	6	0	0
P_3	0	1	1
P_4	4	3	1

→ (4-0) (3-0) (3-2)

Allocation			Work (available)		
A	B	C	A	B	C
0	1	0	3	3	2
2	0	0	5	3	2
3	0	2	7	4	3
2	1	1	7	4	5
0	0	2	7	5	5
			10	5	7

WORKING:

P0 ignored as its need (i.e: (7,4,3)) is greater than its work i.e: (3,3,2)

Step ①:

- Since need of P1 (1,2,2) is lesser than the work vector (i.e: 3,3,2) right now; ∴ move to ② to update work.

Step ②.

Work = work + Allocation[P1]

Work = (3,3,2) + (2,0,0)

Work = (5, 3, 2)

Now add 'P1' in safe sequence.

- After P1 added in safe sequence; P2 is skipped as the need of P2 (6,0,0) is less than current work (5,3,2).
  ∴ we'll move to P3 whose need (i.e: 0,1,1) < work (5,3,2)
  So we'll update work = 7,4,3. P3 added in safe sequence.
- Now after P3; P4 is also added in safe sequence.
- Since P0-P4 all input done so repeat as P0 & P2 aren't in safe sequence till now.
- In second iteration, need of P0 (7,4,3) < work (7,4,5) ∴ P0 in sequence
- Then finally P2 is added in safe sequence as need of P2 (6,0,0) is less than current work = (7,5,5). Updated work = (10, 5, 7)

Answer → Safe Sequence = <P1, P3, P4, P0, P2>

⟹ Actual Banker's Algo / Resource Req. Algo. (it uses Safety Algo).

- A process requests for some resources' instances. That'll be in vector form [1|2|0|3]. (process wants 1 instance of resource A, 2 of resource B, 0 of C & 3 of D)

STEPS:

① If Request <= Need; (agar process kay jo shuru main max. main max need batai thi; uss process nay max say ziada request kr li; only then we'll proceed). go to step ②. Otherwise Invalid req.

② If $Request_i <= Available$, go to ③. Otherwise we don't have enough resources and end the algo.

③ Now we'll pretend to change the state like we've allocated the requested resources to process Pi. This is being done so we get a new state which will then pass through safety Algo.

$Available = Available - Request_i$
$Allocation_{(i)} = Allocation_{(i)} + request$
$Need = Need_i - request$.
} New state with these three.

④ After this, if this new state is passed by safety algo. then we'll actually do all the changes in ③ (which we pretended). So all the resources are actually allocated.

Q: P0 request (0,2,0). 0 instances of A, 2 of B, 0 of C. Should this be granted request?

Banker's Algo:

① Request (P0) ≤ Need(P0) as (0,2,0) ≤ (7,4,3) ✓
② Request (P0) ≤ Available(P0) as (0,2,0) ≤ (3,3,2) ✓

→ Prev. Question table "Available column" of P0.

③ Now we'll pretend to make a new state using the three formulas in step 3 (prev. page).
NOTE: Only P_0 will be update as it's the only one who asked for a new request.
(Table in solution of safety Algo on prev. Page.

	Need			Allocation			Available		
	A	B	C	A	B	C	A	B	C
P_0	7	4	3	0	3	0	3	1	2
P_1	1	2	2	2	0	0	5	2	3
P_2	6	0	0	3	0	2	7	2	3
							10	2	5
							10	5	5

Available = Available − Request
Avail = (3,3,2) − (0,2,0)
= (3,1,2) updated.

Safe sequence = $\langle P_3, P_1, P_2, P_0, P_4 \rangle$

Need = Need$_i$ − request

Need = (7,4,3) − (0,2,0)
(prior page)
Need = (7,2,3) updated

Allocation = Allo. + Request

ALLOC = (0,1,0) + (0,2,0)
= (0,3,0) updated

- Now since we're new row of P_0 with new "available/work;"
∴ we will keep on doing the 3 steps of Safety Algo (prior three pages) to keep updating work/available like (5,2,3), (7,2,3) ...
- This will give us a safe sequence (IF POSSIBLE)

④ IF we find a safe sequence by running the 3 steps again & again on UPDATED state (with new P_0 row); then; we'll actually make the changes we did when pretending in ③.